Autonomous Vehicle Control Using Model Predictive Control

Reproducible Project Report

Bimlendra Ray
University of Texas at Dallas

March 11, 2026

Contents

1 Introduction **1**
1.1 Background . . . 1
1.2 Objective . . . 1
1.3 Contributions . . . 1

2 System Architecture **2**
2.1 Overall System . . . 2

3 Mathematical Model **3**
3.1 Vehicle Kinematic Model . . . 3

4 Controller Design **4**
4.1 Model Predictive Control . . . 4
4.2 Reference Path Generation . . . 5
4.3 Reference Steering Angle Generation . . . 5

5 Perception System **7**
5.1 Objective . . . 7
5.2 Perception Pipeline Overview . . . 7
5.3 Traffic Light Perception . . . 7
5.3.1 Spatial Prior and Candidate Selection . . . 7
5.3.2 Color Classification . . . 8
5.3.3 Traffic Temporal Filtering . . . 8
5.4 Obstacle Perception . . . 8
5.4.1 Region and Lane Prior . . . 8
5.4.2 Depth Gating . . . 8
5.4.3 Obstacle Temporal Filtering . . . 8
5.5 Controller-Perception Interface . . . 9
5.6 Perception Parameters . . . 9
5.7 Reproducibility Notes . . . 9
5.8 Current Limitations and Improvement Directions . . . 9

6 Implementation **10**
6.1 Software Environment . . . 10
6.2 Repository Structure . . . 10
6.2.1 File Roles . . . 11
6.2.2 Execution Flow . . . 11
6.2.3 Why This Structure Helps Reproducibility . . . 11

7 Experimental Setup **12**
7.1 Simulation Environment 12
7.1.1 Execution Setup 12
7.1.2 Evaluation Protocol 12
7.2 Key Parameters 13
7.2.1 Control, mission, and vehicle parameters 13
7.2.2 Cost weights and low-level speed loop 13
7.2.3 Perception and safety-relevant constants 14

8 Results **15**
8.1 Trajectory Tracking 15
8.2 Performance Metrics 15

9 Discussion **17**

10 Reproducibility Guide **18**

11 Conclusion **19**

List of Figures

2.1 System architecture of the QCar NMPC trajectory tracking framework. . 2

8.1 Trajectory Tracking 16

List of Tables

5.1 Perception parameters used in this application 9

6.1 Software environment used for implementation 10

7.1 Core control and mission parameters 13
7.2 MPC cost weights and speed-loop parameters 13
7.3 Perception, traffic handling, and simulation timing constants 14

8.1 Performance results 15

Chapter 1

Introduction

1.1 Background

Autonomous ground vehicles must follow predefined trajectories accurately while respecting vehicle dynamics and control constraints. Trajectory tracking controllers are responsible for minimizing the deviation between the vehicle's actual motion and a desired path.

Traditional control approaches such as PID or geometric controllers can achieve acceptable tracking performance but often struggle to handle nonlinear dynamics, input constraints, and predictive planning. Model Predictive Control (MPC) provides a powerful alternative because it explicitly considers system dynamics and constraints while optimizing control actions over a future prediction horizon.

In this work, a nonlinear model predictive control (NMPC) framework is implemented for the Quanser QCar platform. The controller operates in Frenet coordinates, where the vehicle state is expressed relative to the reference path rather than global Cartesian coordinates. This representation naturally separates longitudinal progress along the path from lateral tracking errors, simplifying the control formulation.

1.2 Objective

The goal of this project is to implement and evaluate a control strategy for autonomous vehicle trajectory tracking.

1.3 Contributions

Contributions in this work include the following.

- Development of a nonlinear vehicle model expressed in Frenet coordinates
- Formulation of an NMPC controller for trajectory tracking
- Integration of a state estimation module for heading estimation
- Generation of reference paths from discrete waypoints using arc-length parameterization

The proposed approach provides an effective framework for autonomous vehicle trajectory tracking in structured environments.

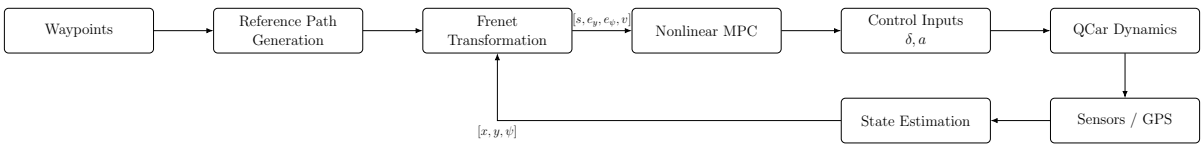

Figure 2.1: System architecture of the QCar NMPC trajectory tracking framework.

Chapter 2

System Architecture

2.1 Overall System

The trajectory tracking system consists of four major components:

- Reference path generation
- Frenet coordinate transformation
- Nonlinear model predictive controller
- Vehicle actuation and feedback

The system architecture is illustrated conceptually in Figure 1.

Chapter 3

Mathematical Model

3.1 Vehicle Kinematic Model

The vehicle is modeled using a kinematic bicycle model, expressed in the Frenet frame attached to the reference path. Unlike Cartesian coordinates, the Frenet frame naturally separates longitudinal progress along the path from lateral deviation, which simplifies trajectory tracking.

The state vector is defined as $\mathbf{x} = [s, e_y, e_\psi, v]^\top$, where:

- s: longitudinal position along the reference path.
- e_y: lateral deviation from the path centerline.
- e_ψ: heading error relative to the path tangent.
- v: vehicle forward speed.

The path curvature is denoted by $\kappa(s)$, which is a known function of the reference trajectory.

$$\begin{aligned}
\dot{s} &= \frac{v\cos(e_\psi)}{1-\kappa\, e_y} \\
\dot{e}_y &= v\sin(e_\psi) \\
\dot{e}_\psi &= \frac{v}{L}\tan(\delta) - \kappa\,\dot{s} \\
\dot{v} &= a
\end{aligned}$$

whereas, $\dot{s}$ describes how fast the vehicle progresses along the reference path. $\dot{e_y}$ governs how lateral deviations evolves. $\dot{e_\psi}$ describes how heading error evolves over time. $\dot{v}$ is directly dependent on the longitudinal acceleration.

Chapter 4

Controller Design

4.1 Model Predictive Control

The NMPC controller minimizes a cost function that balances tracking accuracy, control effort, and smoothness. The stage cost for this is given below:

$$J = \sum_{k=0}^{N-1} \Big(w_{e_y} e_{y,k}^2 + w_{e_\psi} e_{\psi,k}^2 + w_v (v_k - v_k^{\text{ref}})^2 + w_\delta (\delta_k - \delta_k^{\text{ref}})^2 + w_a a_k^2 \Big) + \sum_{k=1}^{N-1} \Big(w_{\dot{\delta}} \dot{\delta}_k^2 + w_{\dot{a}} \dot{a}_k^2 \Big)$$

$$W = \text{diag}\big(w_{e_y},\, w_{e_\psi},\, w_v,\, w_\delta,\, w_a,\, w_{\Delta\delta},\, w_{\Delta a}\big) = \text{diag}(50,\, 50,\, 0.02,\, 1,\, 2,\, 20,\, 0.2)\,.$$

Where, w_{e_y} and w_{e_ψ} penalize lateral and heading errors. $v_k - v_k^{ref}$ enforces the speed tracking and w_v penalizes the error in the velocity. w_{δ_k} and w_{α_k} penalize aggressive steering and acceleration. $w_{\dot{\delta}}$ and $w_{\dot{\alpha}}$, these terms penalizes the rapid changes in steering and acceleration. The terminal cost is same as the stage cost and it helps for the closed loop stability.

$$\dot{\delta}_k = \delta_k - \delta_{k-1}, \quad k = 1, \ldots, N-1$$
$$\dot{a}_k = a_k - a_{k-1}, \quad k = 1, \ldots, N-1$$

$\dot{\delta}_k$ gives the rate of change of steering with respect to the sampling time. $\dot{\alpha}_k$ gives rate of change of acceleration.

The state constraint for e_y and e_ψ is required to follow trajectory and v_y to keep the speed within the desired speed of the QCar.

$$-e_{y,\max} \le e_{y,k} \le e_{y,\max}, \quad k = 0, \ldots, N$$
$$v_{\min} \le v_k \le v_{\max}, \quad k = 0, \ldots, N$$

The input constraint δ_k is α_k models the QCar hardware constraints whereas $\dot{\delta}_k$ prevents the rapid change in steering and for numerical stability.

Lateral error bound:

$$-e_{y,\max} \le e_{y,k} \le e_{y,\max} e_{y,\max} = \frac{W}{2} - b = \frac{0.20}{2} - 0.01 = 0.09 \text{ m}$$

Steering input bound:

$$-\delta_{\max} \leq \delta_k \leq \delta_{\max}\delta_{\max} = \frac{\pi}{6} \approx 0.524 \text{ rad}$$

Acceleration input bound:

$$a_{\min} \leq a_k \leq a_{\max}a_{\min} = -1.5, \quad a_{\max} = 1.5 \text{ m/s}^2$$

Speed state bound:

$$v_{\min} \leq v_k \leq v_{\max}v_{\min} = 0.001, \quad v_{\max} = 1.0 \text{ m/s}$$

Steering rate bound:

$$\begin{aligned}
&\delta_k - \delta_{k-1}| \leq \dot{\delta}_{\max}T_s \\
&\dot{\delta}_{\max} = 2.0 \text{ rad/s}, \quad T_s = \frac{1}{30} \text{ s} \\
&|\delta_k - \delta_{k-1}| \leq 2.0 \cdot \frac{1}{30} = 0.0667 \text{ rad/step}
\end{aligned}$$

Acceleration rate bound:

$$\begin{aligned}
&|a_k - a_{k-1}| \leq \dot{a}_{\max}T_s \\
&\dot{a}_{\max} = 2.0 \text{ m/s}^3, \quad T_s = \frac{1}{30} \text{ s} \\
&|a_k - a_{k-1}| \leq 2.0 \cdot \frac{1}{30} = 0.0667 \text{ (m/s}^2\text{)/step}
\end{aligned}$$

4.2 Reference Path Generation

The reference path is generated from discrete waypoints and parameterized by arc length. The waypoints $w_i = (x_i, y_i)$ is generated from the path sequence. The path length, s is computed from the waypoints. Arc length parameterization creates a smooth longitudinal parameter s independent of waypoint spacing. And, then the curvature of the path is computed using the spline derivative.

$$\begin{aligned}
&\Delta x_i = x_{i+1} - x_i, \quad \Delta y_i = y_{i+1} - y_i \\
&\Delta s_i = \sqrt{\Delta x_i^2 + \Delta y_i^2} \\
&s_0 = 0, \quad s_i = \sum_{j=0}^{i-1} \Delta s_j
\end{aligned}$$

4.3 Reference Steering Angle Generation

$$\begin{aligned}
\text{Horizon time index:} \quad & t_k = k\,T_s, \qquad k = 0, \ldots, N \\
\text{Predicted arc-length samples:} \quad & s_k = s_0 + t_k \max(v_{\text{meas}}, 0) \\
\text{Reference curvature sequence:} \quad & \kappa_k^{\text{ref}} = \kappa_{\text{path}}(s_k), \qquad k = 0, \ldots, N-1 \\
\text{Current curvature reference:} \quad & \kappa^{\text{ref}} = \kappa_0^{\text{ref}}
\end{aligned}$$

Steering reference from curvature: $\delta_k^{\mathrm{ref}} = \arctan\big(L\,\kappa_k^{\mathrm{ref}}\big), \qquad k = 0, \ldots, N-1$

If curvature-based speed reference is enabled:

$$v_k^{\mathrm{ref}} = \mathrm{clip}\left(\frac{v_{\mathrm{base}}}{1 + g_\kappa\,|\kappa_k^{\mathrm{ref}}|},\, v_{\mathrm{min}},\, v_{\mathrm{max}}\right), \qquad k = 0, \ldots, N$$

Chapter 5

Perception System

5.1 Objective

The perception module provides real-time scene understanding for safe autonomous driving in QLabs. Its purpose is to estimate:

- traffic light state: {`RED, YELLOW, GREEN, UNKNOWN, NONE`}
- obstacle presence: `True/False`
- obstacle distance (if valid depth is available)

These outputs are consumed by the control loop to enforce stop/go behavior and safety constraints.

5.2 Perception Pipeline Overview

The perception stack is implemented in the `RacingPerception` class and runs continuously with camera updates. The processing pipeline is:

1. Acquire synchronized RGB and depth frames from `QCar2DepthAligned`.
2. Detect candidate regions using `ObjectDetection`.
3. Classify traffic-light color using HSV masks inside a selected bounding box.
4. Detect obstacles using lower-image priors and depth-gated filtering.
5. Apply temporal filtering (persistence counters) to reduce flicker.
6. Publish `(traffic_state, obstacle, distance, timestamp)` to shared state.

5.3 Traffic Light Perception

5.3.1 Spatial Prior and Candidate Selection

Only detections labeled as `traffic` are considered, and candidates are filtered by image location:

- upper ROI focus for traffic lights
- rejection of candidates with low geometric plausibility

5.3.2 Color Classification

Inside the selected traffic-light bounding box:

- top-centered crop is extracted to reduce road/background influence
- ROI is converted to HSV
- the crop is split into three vertical bands:
  - top band: red
  - middle band: yellow
  - bottom band: green
- per-band active-pixel counts and ratios are computed

The dominant valid band determines the observed state. If score thresholds are not met, state is set to `UNKNOWN`.

5.3.3 Traffic Temporal Filtering

To suppress frame-level misclassification, persistence logic is used:

state changes only if the same observation is confirmed for n_{confirm} frames.

In this project:

$$n_{\text{confirm}} = 2, \quad n_{\text{decay}} = 1.$$

5.4 Obstacle Perception

5.4.1 Region and Lane Prior

Obstacle reasoning focuses on lower image regions where near-road objects appear. A lane-centered gate rejects detections far from the ego-lane center in image coordinates.

5.4.2 Depth Gating

Depth-aligned measurements are used to estimate obstacle distance:

- valid depth range filtering
- median depth over segmented obstacle region
- optional stopping gate based on distance threshold

5.4.3 Obstacle Temporal Filtering

Obstacle events are latched by persistence counters:

$$\text{obstacle} = 1 \quad \Leftrightarrow \quad c_{\text{obs}} \geq n_{\text{obs,confirm}}.$$

This avoids rapid stop/release switching under noisy detections.

5.5 Controller-Perception Interface

The control loop reads shared perception output and applies:

- stale-data rejection (`age` > `PERCEPTION_STALE_S`)
- light-based stop policy (`RED/YELLOW` → stop)
- obstacle-based stop policy
- hold timers (`RED_HOLD_S`, `OBS_HOLD_S`) for debounce

This interface is safety-first: uncertain or stale signals are handled conservatively.

5.6 Perception Parameters

Table 5.1: Perception parameters used in this application

Parameter	Meaning	Value	Role
`obs_depth_min/max_valid`	Depth validity interval	0.03–10.0 m	Reject bad depth
`obs_min_contour_area`	Min mask area	4500 px	Noise suppression
`light_confirm_frames`	Light persistence	2	Temporal stability
`obs_confirm_frames`	Obstacle persistence	2	Temporal stability
`hsv_red1`	Red HSV range 1	(0,120,80)–(10,255,255)	Lamp segmentatio
`hsv_red2`	Red HSV range 2	(170,120,80)–(180,255,255)	Hue wrap-around
`hsv_yellow`	Yellow HSV range	(18,120,80)–(38,255,255)	Lamp segmentatio
`hsv_green`	Green HSV range	(48,100,90)–(95,255,255)	Lamp segmentatio

5.7 Reproducibility Notes

To reproduce perception results consistently:

- keep simulator lighting/environment unchanged
- keep camera placement and intrinsics fixed
- keep HSV thresholds and persistence counters unchanged
- report software commit hash and all parameter overrides

5.8 Current Limitations and Improvement Directions

- HSV thresholds can be sensitive to illumination changes.
- Small/far obstacles may produce unstable depth estimates.
- Single-camera geometry can limit occlusion handling.

Future work can include learned traffic-light classifiers, depth confidence modeling, and multi-sensor fusion for more robust obstacle decisions.

Chapter 6

Implementation

6.1 Software Environment

All simulations and experiments were conducted using a Python-based robotics development environment integrated with the Quanser QLabs autonomous vehicle simulator. The implementation was executed on a standard workstation running a modern operating system.

The primary programming language used for development was Python, due to its extensive ecosystem of scientific computing libraries and optimization frameworks. The control algorithms were implemented using optimization tools designed for real-time control applications.

Table 6.1 summarizes the main software components used in this project.

Table 6.1: Software environment used for implementation

Component	Tool / Framework	Version
Operating System	Windows 11	–
Programming Language	Python	3.13
Scientific Computing	NumPy, SciPy	Latest
Control Framework	CasADi / ACADOS	Latest
Simulator	Quanser QLabs	2025
Visualization	Scopes	Latest
Version Control	Git	2.x
IDE / Editor	Visual Studio Code	Latest

The Quanser QLabs simulator provides a realistic virtual environment for autonomous vehicle development, allowing integration of perception, planning, and control algorithms without requiring physical hardware. The optimization framework CasADi was used for formulating nonlinear optimal control problems, while ACADOS was used for efficient real-time MPC computation.

6.2 Repository Structure

This project is organized as a small, modular codebase where each file has one clear responsibility. The separation makes it easier for future students to debug, replace modules,

and extend the controller.

```
project/

 main.py
 Setup_Real_Scenario.py
 Path2D.py
 FrenetNonLinearMPC.py
 FrenetNonLinearMPCController.py
```

6.2.1 File Roles

- `main.py`: Entry point of the project. It initializes the scenario, creates the path and controller objects, runs the simulation/control loop, and stores or prints results.
- `Setup_Real_Scenario.py`: Scenario definition module. It contains environment setup, initial vehicle state, obstacle/traffic settings.
- `Path2D.py`: Path generation and geometric utilities. It defines the reference path in 2D and provides operations needed by the controller (e.g., path sampling, heading/curvature access, nearest-point logic).
- `FrenetNonLinearMPC.py`: Core nonlinear MPC formulation in Frenet coordinates. It defines states/inputs, prediction model, cost function, and constraints, and builds the optimization problem solved at each control step.
- `FrenetNonLinearMPCController.py`: Controller wrapper/interface. It connects the MPC solver to the simulation loop, updates measurements and references online, calls the solver, and returns control commands.

6.2.2 Execution Flow

1. `main.py` loads scenario settings from `Setup_Real_Scenario.py`.
2. A reference path is created using `Path2D.py`.
3. The MPC problem is initialized from `FrenetNonLinearMPC.py`.
4. `FrenetNonLinearMPCController.py` runs the closed-loop control at each time step.
5. `main.py` logs trajectories, control actions, and performance metrics.

6.2.3 Why This Structure Helps Reproducibility

- Scenario settings are isolated, so experiments can be repeated by reusing the same setup file.
- Path logic and control logic are separated, making parameter changes safer and easier to track.
- The MPC formulation is centralized, so future improvements (new constraints, new cost terms) can be implemented without rewriting the whole project.

Chapter 7

Experimental Setup

7.1 Simulation Environment

All experiments are carried out in the Quanser QLabs virtual environment with the QCar platform. QLabs provides a controlled and repeatable setup for testing trajectory tracking and traffic interaction without hardware risk. The simulator includes realistic lane geometry, vehicle kinematics, and traffic elements (e.g., static obstacles and moving actors), which are required to evaluate controller behavior under constrained urban-like driving scenarios.

For each run, the same map, initial vehicle pose, and controller configuration are used unless stated otherwise.

7.1.1 Execution Setup

- Simulator: Quanser QLabs
- Vehicle model: QCar2
- Control architecture: Nonlinear MPC in Frenet coordinates
- Loop type: Closed-loop simulation (state feedback at each control step)
- Logging: vehicle states, reference states, control inputs, and tracking errors

7.1.2 Evaluation Protocol

Each experiment is repeated under identical controller settings, and performance is assessed using:

- Lateral tracking error
- Heading error
- Speed tracking quality

When multiple runs are reported, metrics are presented as mean and standard deviation.

7.2 Key Parameters

Table 7.1 and Table 7.3 summarize the main constants used in this application. All values below are taken from the implementation and should be kept unchanged for strict reproducibility.

7.2.1 Control, mission, and vehicle parameters

Table 7.1: Core control and mission parameters

Symbol / Name	Meaning	Value	Notes
T_s	Controller sampling time	$1/30 \approx 0.0333$ s	From `controllerUpdateRate` =
N	MPC prediction horizon	30 steps	Prediction window $NT_s \approx 1.0$ s
v_{ref}	Reference speed	1.0 m/s	Also used as $v_{\max}$ in MPC
$v_{\min}$	Minimum speed in MPC	0.001 m/s	Avoids singular low-speed beha
$\delta_{\max}$	Steering limit	$\pi/6$ rad ($\approx 30°$)	Input constraint
$\dot{\delta}_{\max}$	Steering-rate limit	2.0 rad/s	Smooth steering change
$a_{\max}$	Acceleration bound	1.5 m/s^2	$a_{\min} = -1.5$, $a_{\max} = +1.5$
$\dot{a}_{\max}$	Acceleration-rate limit	2.0 m/s^3	Input-rate smoothing
l_f, l_r	Vehicle geometry	0.128 m, 0.128 m	Wheelbase $L = l_f + l_r = 0.256$
W	Lane width	0.20 m	Frenet lateral bound setup
b	Safety buffer	0.01 m	Tightens lateral bounds
$e_{y,\min}, e_{y,\max}$	Lateral error bounds	$[-0.09, +0.09]$ m	$e_y = \pm(W/2 - b)$
$a_{\text{lat,max}}$	Lateral accel for speed profile	1.5 m/s^2	Path speed shaping
`HUB_NODE`	Hub graph node	10	Mission route anchor
`PICKUP_NODE`	Pickup graph node	20	Mission route anchor
`DROPOFF_NODE`	Drop-off graph node	9	Mission route anchor
`GOAL_TOL`	Goal tolerance radius	0.18 m	Goal reached condition
`STOP_DURATION`	Stop time at pickup/drop-off	2.0 s	FSM dwell time

7.2.2 Cost weights and low-level speed loop

Table 7.2: MPC cost weights and speed-loop parameters

Parameter	Value	Interpretation
α	20	Global scaling for state weights
$Q = \alpha[0.001,\ 3,\ 2.5,\ 0.001]$	$[0.02,\ 60,\ 50,\ 0.02]$	Emphasizes e_y and e_ψ tracking
β	2	Global scaling for input weights
$R = \beta[0.5,\ 1]$	$[1,\ 2]$	Penalizes steering and accel effort
γ	20	Global scaling for input-rate weights
$S_{\Delta u} = \gamma[1,\ 0.01]$	$[20,\ 0.2]$	Encourages smooth actuation
PID (K_p, K_i, K_d)	$(0.2,\ 1.0,\ 0.0)$	Low-level speed actuation loop

7.2.3 Perception and safety-relevant constants

Table 7.3: Perception, traffic handling, and simulation timing constants

Name	Meaning	Value	Notes
`PERCEPTION_RATE_HZ`	Perception update rate	10 Hz	Shared-state updates
`PERCEPTION_STALE_S`	Stale data timeout	0.25 s	Ignore outdated detections
`RED_HOLD_S`	Red/yellow hold timer	0.50 s	Debounce stop/go switching
`OBS_HOLD_S`	Obstacle hold timer	0.40 s	Debounce obstacle events
`OBS_DISTANCE_STOP_M`	Obstacle stop gate	1.20 m	Stop decision gate in contro
`obs_stop_distance_m`	Perception obstacle horizon	12.0 m	Detection-stage distance hor
`obs_depth_min/max_valid`	Valid depth range	0.03–10.0 m	Reject invalid depth values
`obs_min_contour_area`	Obstacle mask threshold	4500 px	Suppresses small noise blobs
`light_confirm_frames`	Traffic light persistence	2 frames	Temporal filtering
`obs_confirm_frames`	Obstacle persistence	2 frames	Temporal filtering
`GREEN_TIME`	NS/EW green phase	5 s	In `qlabs_setup.py`
`YELLOW_TIME`	NS/EW yellow phase	2 s	In `qlabs_setup.py`
`RED_CLEAR`	All-red clearance phase	1 s	In `qlabs_setup.py`

Reproducibility note. The implemented values differ from generic examples often used in drafts (e.g., $T_s = 0.02$ s, $N = 20$, $v_{\max} = 2$ m/s). For this project, the actual settings are $T_s \approx 0.0333$ s, $N = 30$, and $v_{\max} = v_{\text{ref}} = 1.0$ m/s.

Chapter 8

Results

8.1 Trajectory Tracking

Trajectory tracking is generally strong: the lateral error remains within about $|e_y| < 0.05\,\mathrm{m}$, and the heading error is typically below $|\epsilon_\psi| < 1.5$ (as reported in Fig. 8.1). However, Fig. 8.1 also shows a noticeable mismatch in velocity tracking.

This behavior follows from the current control architecture. The reference speed v_{ref} is generated from the path-curvature profile (approximately 0 to 1.8 m/s). The MPC then optimizes acceleration, and this acceleration is converted into the final speed command sent to the QCar. For steering, the controller uses both the MPC steering output δ and a steering reference δ_{ref} computed online from path curvature.

Using curvature-based δ_{ref} improves nominal path-following and helps compensate steering lag caused by system and computation delays. For this reason, a steering-error term is included in the MPC cost. The trade-off is reduced robustness when the vehicle deviates from the nominal path: because δ_{ref} depends strongly on local curvature, the controller can overreact and produce abrupt steering corrections, which may destabilize the vehicle. This limitation is addressed in future work by introducing a more state-dependent steering reference and a smoother recovery strategy for large tracking offsets.

8.2 Performance Metrics

Metric	Value	Unit
RMSE position	0.12	m
Max error	0.35	m
Compute time	8.5	ms

Table 8.1: Performance results

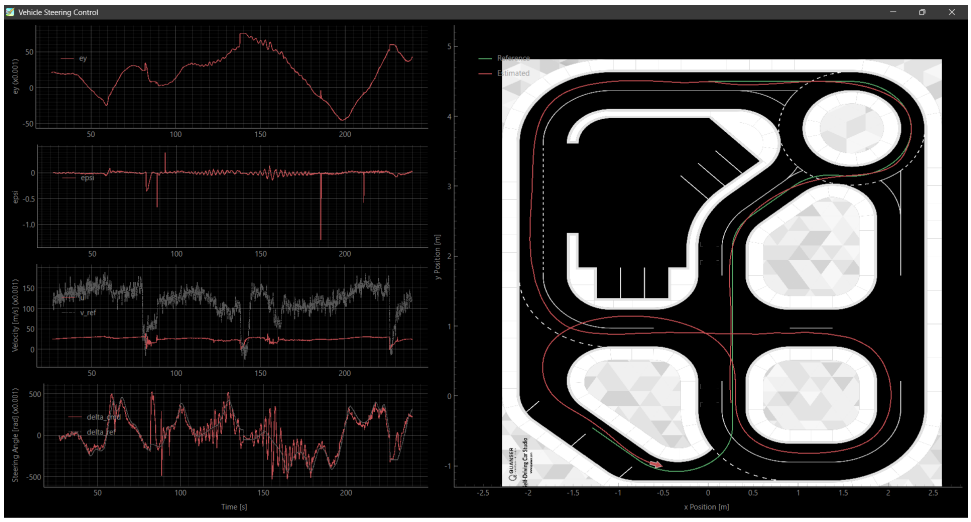

Figure 8.1: Trajectory Tracking

Chapter 9

Discussion

The results demonstrate that the proposed control strategy achieves stable trajectory tracking with acceptable computational load.

Future improvements include

- improved perception integration
- obstacle avoidance
- robust MPC formulation

Chapter 10

Reproducibility Guide

To reproduce the experiments:

```
git clone https://github.com/bhimray/Quanser_1st_stage_submission.git
pip install -r requirements.txt
python main.py
```

Expected outputs include trajectory tracking plots and performance metrics reported in Chapter 5.

Chapter 11

Conclusion

This work presented a reproducible framework for autonomous vehicle trajectory tracking using Model Predictive Control.

The controller successfully achieved stable tracking while maintaining real-time computational performance.

Bibliography

[1] Rawlings, J. B. *Model Predictive Control*. MIT Press, 2017.

[2] Rajamani, R. *Vehicle Dynamics and Control*. Springer, 2011.